Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Softvér na identifikáciu plagiátov technickej dokumentácie

Príloha B: Používateľska príručka

Bakalárska práca

2025 **Taras Hamkalo**

Obsah

1 Použitie programu na porovnanie dokumentov **1**
1.1 Programové súbory 1
1.2 Systémové požiadavky 1
1.3 Preklad programu 2
1.4 Príklady vstupu pre program 2
1.5 Spustenie 3
1.6 Vystup 3
1.7 Konfigurácia 3

2 Použitie používateľského rozhrania **6**
2.1 Inštalácia programu 6
2.1.1 Všeobecná inštalácia na webový server 6
2.1.2 Inštalácia na HRON 7
2.1.3 Inštalácia a spustenie pomocou Dockeru 8
2.2 Začiatok práce s rozhraním a definícia vstupu 8
2.3 Stránka „Sumár analýzy“ 9
2.4 Práca s tabuľkami 9
2.5 Stránka so zobrazením konkrétneho dokumentu 10
2.6 Stránka pre zobrazenie dvojice dokumentov 12
2.7 Zobrazenie teplotnej mapy a rozdiel v obsahu 14
2.8 Zobrazenie grafu odovzdaní 14

Zoznam obrázkov

2.1 Obrázok stránky „Import analýzy“ 8
2.2 Obrázok stránky „Sumár analýzy“ 9
2.3 Príklad tabuľky a zvýraznenie funkcií 10
2.4 Obrázok stránky so zobrazením konkrétneho dokumentu 11
2.5 Obrázok stránky dvojice dokumentov, prvá časť 13
2.6 Obrázok stránky dvojice dokumentov, druhá časť 13
2.7 Obrázok stránky pre zobrazenie teplotnej mapy 15
2.8 Obrázok stránky pre zobrazenie rozdielu v obsahu 15
2.9 Obrázok stránky pre zobrazenie grafu odovzdaní 16

Zoznam tabuliek

1.1 Opis vstupných parametrov nástroja `plagdet-cli` 4

2.1 Tabuľky a vyhľadávacie polia 10
2.2 Exportované polia metaúdajov 11
2.3 Legenda pre farby vyskytujúce sa v pri prehľade metaúdajov 12
2.4 Legenda pre farby vyskytujúce sa v textovom editore 12
2.5 Exportované polia dvojíc dokumentov 12
2.6 Legenda pre farby vyskytujúce sa v textovom editore pri prehľade dvojice dokumentov 14

1 Použitie programu na porovnanie dokumentov

Program na porovnanie dokumentov je lokálny nástroj napísaný v jazyku Java. Umožňuje vám zadať množinu prác, ktoré majú byť aktuálne analyzované, a zároveň poskytuje možnosť vytvoriť index dokumentov a použiť ho na zrýchlenie porovnávania s množinou potenciálnych zdrojov plagiátorstva.

Po zadaní vstupu program začne kontrolu, pričom zobrazuje priebeh na štandardný výstup. Po ukončení analýzy program uloží výsledok do archívu vo formáte `zip`.

1.1 Programové súbory

- `pladet-cli-3.jar` – JAR súbor obsahujúci všetky komponenty a zdroje na vykonanie programu.
- `report.zip` – Štandardný názov výstupného archívu s výsledkami, ak už existuje, bude prepísaný. Používateľ môže zadať vlastný názov pri spustení programu.
- `logs` – Adresár, ktorý sa vytvorí pri spustení programu, obsahuje logy programu na účely diagnostiky.

1.2 Systémové požiadavky

Program je napísaný v Jave, verzia 17. Na systéme teda musí byť nainštalovaná Java 17 alebo novšia.

Základná verzia nástroja `pladget-cli-3.jar` je optimalizovaná na efektívnu prácu v systéme Linux – závislosti pre Windows a macOS sú vypnuté.

Na spustenie programu je potrebné mať nainštalovanú zdieľanú knižnicu morfologického anotátora MorphoDiTa. Túto knižnicu môžete zadať aj pri spustení

programu ako parameter Javy.

Kópiu knižnice nájdete v adresári „libs“ projektu, pod názvom „libmorphodita_java.so“, prípadne si ju stiahnite podľa inštrukcií na stránke[1].

1.3 Preklad programu

Na preklad programu vykonajte nasledujúce kroky[2]:

```
1. git clone {repository-url}
2. cd plagiarism-detection
3. ./gradlew :plagdet-cli:shadowJar
```

Klonovanie a zostavenie JAR súboru môže trvať pomerne dlho. Na testovanom systéme zostavenie trvá približne 1.5 minúty. Po dokončení tohto procesu sa v koreňovom adresári projektu vytvorí adresár „dist“, v ktorom sa bude nachádzať súbor JAR aplikácie.

1.4 Príklady vstupu pre program

1: Najjednoduchší vstup je jeden dokument v podporovanom formáte:

```
file.docx
file.pdf
file.doc
```

2: Možno zadať aj množinu dokumentov ako adresár alebo ZIP archív:

```
a/      -> [file.docx, file.pdf, file.doc]
a.zip   -> [file.docx, file.pdf, file.doc]
a.zip   -> [a/file.docx, b/file.pdf, c/file.doc]
```

3: Program podporuje aj vnorené ZIP archívy, keď obsahujú súbory vo formáte podľa príkladov vyššie. Príklady vstupu s archívmi sú:

```
a.zip   -> [a.zip, b.zip, c.zip]
a/      -> [a.zip, b.zip, c.zip]
```

[1] https://ufal.mff.cuni.cz/morphodita/install#java_installation

[2] Ako repozitár môžete použiť git@git.kpi.fei.tuke.sk:kpi-zp/2025/bp.taras.hamkalo/plagiarism-detection.git

1.5 Spustenie

Dôležité: Ak knižnica MorphoDiTa nie je nainštalovaná globálne, môžete ju zadať nasledovne pred spustením nástroja:

```
java -Dsystem.library.path={cesta} -jar plagdet-cli-3.jar X
```

1: Spustenie kontroly dokumentov:

```
java -jar plagdet-cli-3.jar -n {vstup}
```

2: Spustenie s vlastnou cestou k výstupnému súboru:

```
java -jar plagdet-cli-3.jar -n {vstup} --output-path /cesta/k/reportu
```

3: Spustenie za účelom indexovania dokumentov:

```
java -jar plagdet-cli-3.jar -n {vstup} -i -l {cesta_k_indexu}
```

4: Spustenie s použitím existujúceho indexu:

```
java -jar plagdet-cli-3.jar -n {vstup} -l {cesta_k_indexu}
```

1.6 Vystup

Po úspešnom dokončení programu bude v adresári, z ktorého bol program spustený, vytvorený súbor vo formáte ZIP (predvolené meno je `report.zip`). Tento súbor obsahuje všetky informácie potrebné na vizualizáciu a teda obsah dokumentov, metaúdaje a výsledky porovnania.

1.7 Konfigurácia

V prípade potreby špecifických parametrov (napr. zmena textu úlohy, úprava prahov podobnosti) ich možno zadať pomocou argumentov programu.

Tabuľka 1.1: Opis vstupných parametrov nástroja `plagdet-cli`

Parameter	**Opis**
`-l, --lucene-path <arg>`	Cesta k Lucene indexu, vytvorí sa dočasný, ak nie je zadaný
`-t, --template-path <arg>`	Cesta k šablóne dokumentu
`--enable-metadata-filter`	Pokračuje v porovnávaní aj po zhode autorov/upravovateľov
`--bypass-shingle-filter`	Obíde filtrovanie na základe prekrytia shinglov
`--bypass-length-filter`	Obíde filtrovanie na základe pomeru v počte viet
`--bypass-template-removal`	Nevylučuje zhody so šablónou (predvolené znenie zadania)
`--max-gap <arg>`	Maximálna vzdialenosť medzi zhodami na ich spojenie
`--min-fragment-size <arg>`	Minimálny počet viet na fragment
`--min-fragment-sim <arg>`	Minimálna podobnosť fragmentu
`--sentence-size-ratio-th <arg>`	Zhody s nižším pomer v dĺžke vety sa zahodia
`--alignment-th <arg>`	Prah podobnosti podľa zarovnania slov
`--vector-sim-th <arg>`	Prah vektorovej podobnosti pre výber viet z indexu
`--min-sentences-th <arg>`	Minimálny počet viet na porovnanie pri dopyte indexu
`--sentences-percentage-th <arg>`	Percento indexu zloženého z viet jedného dokumentu, ktoré budú pri dopyte zvážené
`--sentences-percentage-multi-th <arg>`	Maximalné percento viet indexu, ktoré sa zvažia pri dotazoch voči indexu z viacerých dokumentov
`--sentences-per-document-th <arg>`	Počet viet na dokument pri dotaze z viacerých dokumentov
`--shingle-size <arg>`	Veľkosť shinglov
`--shingle-overlap <arg>`	Prekrytie medzi shinglami

pokračovanie na ďalšej strane

Parameter	**Opis**
`--plagiarized-level <arg>`	Prah Jaccardovej podobnosti shinglov na označenie dvojice ako plagiát
`--discard-level <arg>`	Prah Jaccardovej podobnosti shingle pod ktorý sa pár zahodí
`--max-results <arg>`	Max. počet výsledkov pre dokument
`--min-result-sim <arg>`	Minimálna podobnosť na uloženie dvojice do výsledkov
`--max-meta-docs <arg>`	Max. počet dokumentov, ktoré sa získajú z indexu podľa metaúdajov
`--max-sim-docs <arg>`	Max. počet dokumentov, ktoré sa získajú z indexu podľa podobnosti obsahu
`--document-size-ratio-th`	Porovnanie sa neuskutoční ak je pomer veľkostí pod danou prahovou hodnotou (opak odchýlky vo velikosti)
`--min-char-len`	Minimálny počet znakov na uloženie fragmentu ako plagiát

2 Použitie používateľského rozhrania

Používateľské rozhranie je implementované ako klientská webová aplikácia, ktorá poskytuje množstvo funkcií na vizualizáciu a interpretáciu výsledkov kontroly plagiátorstva.

Na začatie práce s touto aplikáciou je potrebné umiestniť ju medzi projekty vášho vybraného webového servera. Ak ste to už urobili, môžete pokojne preskočiť niekoľko nasledujúcich sekcií.

2.1 Inštalácia programu

Program je napísaný vo webovom rámci Angular a vyžaduje Node.js na jeho zostavenie. Na inštaláciu Node.js navštívte stránku [3] a postupujte podľa ich pokynov. Po inštalácii Node.js vykonajte kroky opísané v jednej zo sekcií nižšie, podľa vášho prípadu použitia.

Vo všetkých prípadoch môžete ako odkaz na git repozitár využiť nasledovný odkaz [4].

2.1.1 Všeobecná inštalácia na webový server

Všeobecná inštalácia na webový server môže byť vykonaná podľa nasledujúcich krokov:

```
1. git clone {repository-url}
2. cd ux-taras-hamkalo
3. npm install
4. npm install -g @angular/cli
5. ng build --base-href {html_endpoint_name} \
            --deploy-url {html_endpoint_name}
6. cp ~/dist/plagdet/browser/* {web_server_html_directory}
```

[3] https://nodejs.org/en/download

[4] git@git.kpi.fei.tuke.sk:kpi-zp/2025/bp.taras.hamkalo/ux-taras-hamkalo.git

2.1.2 Inštalácia na HRON

Predpokladom inštalácie programu na HRON je vytvorenie účtu za odkazom [5]. Po vytvorení účtu, HRON poskytuje každému používateľovi možnosť zverejňovať webové stránky v mene používateľa.

Na zverejnenie sa pripojte na server HRON pomocou vami vybraného spôsobu, napríklad `ssh`. Vo vašom domovskom adresári vytvorte adresár `public_html`, ktorý slúži ako koreň pre všetky vaše webové publikácie. Akýkoľvek obsah tohto adresára bude dostupný na adrese vo formáte:

```
hron.fei.tuke.sk/~{hron-login}/{content-path}
```

`hron-login` predstavuje váš login na server HRON. `content-path` je cestou k obsahu, ktorý chcete zobraziť, uvedená relatívne od adresára `public_html`.

Po vytvorení adresára `public_html` vytvorte v ňom adresár, pod ktorým by ste chceli pristupovať k aplikácii, zapamätajte si ho, ďalej ho označujeme ako premennú `target-location`.

Pre zostavenie aplikácie vykonajte nasledujúce kroky:

```
1. git clone {repository-url}
2. cd ux-taras-hamkalo
3. npm install
4. npm install -g @angular/cli
5. ng build --base-href /~{hron-login}/{target-location}/ \
            --deploy-url /~{hron-login}/{target-location}/
```

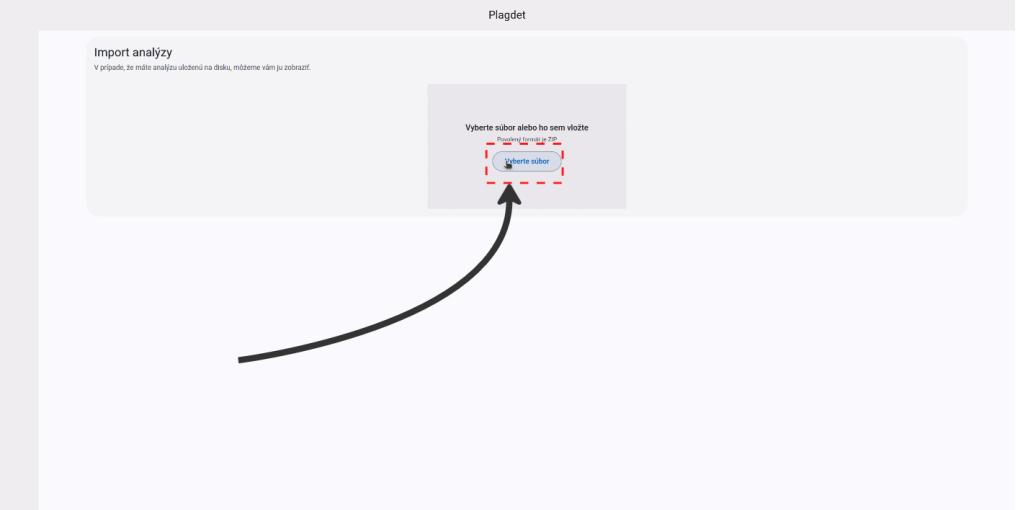
Po zostavení sa vytvorí adresár `dist` v adresáre projektu, v ktorom sa nachádza zostavený program. Na umiestnenie programu do `target-location` vykonajte nasledujúci príkaz z adresáru projektu:

```
cp ./dist/plagdet/browser/* ~/public\_html/{target-location}
```

V prípade že zostavovanie programu sa vykonalo mimo HRON-u, vykonajte príkaz:

```
scp -r ./dist/plagdet/browser/* \
  {hron-login}@hron.fei.tuke.sk:~/public\_html/{target-location}
```

[5] `https://hron.fei.tuke.sk/`



Obrázok 2.1: Obrázok stránky „Import analýzy“

2.1.3 Inštalácia a spustenie pomocou Dockeru

Najjednoduchším spôsobom spustenia zostáva Docker. V projekte je pridaný `Dockerfile`, ktorý používa `nginx` ako implementáciu webového servera.

Na zostavenie a spustenie obrazu vykonajte nasledujúce kroky:

```
1. git clone {repository-url}
2. cd ux-taras-hamkalo
3. docker build -t plagdet-ui .
4. docker run -p 80:80 plagdet-ui
```

Program bude dostupný na porte 80 vášho stroja.

2.2 Začiatok práce s rozhraním a definícia vstupu

Na to aby začať pracovať s rozhraním vytvorte súbor analýzy plagiatú pomocou programu opísaného v kapitole 1. Použite jeden z uvedených spôsobov spustenia, ktorý po dokončení zapíše súbor analýzy na disk.

Uložte cestu k tomuto súboru a zadajte ju pri výbere na stránke „Import analýzy“, obrázok 2.1. Daná stránka je prvou stránkou, na ktorú sa dostanete, takže jej nájdenie nemalo by byt problémom.

Po výbere súboru sa zobrazí okno potvrdenia pre prípadnú kontrolu spratnosti vybraného súboru. Pre jeho skutočne načítania vyberte možnosť „Potvrdiť“.



Obrázok 2.2: Obrázok stránky „Sumár analýzy“

2.3 Stránka „Sumár analýzy“

Táto stránka neobsahuje nejaké kľúčové funkcie pre používateľa, ale zobrazuje všeobecnú charakteristiku analyzovaných súborov. Medzi nimi sú počty súborov, dátum vykonania analýzy a histogram rozdelenia prác podľa najvyššieho nájdeného percenta podobnosti. Príklad stránky je uvedený na obr. 2.2

Z tejto stránky sa tiež otvára prístup k ďalším funkciám systému. K nim môžete prejsť pomocou navigačného panela na ľavej strane rozhrania.

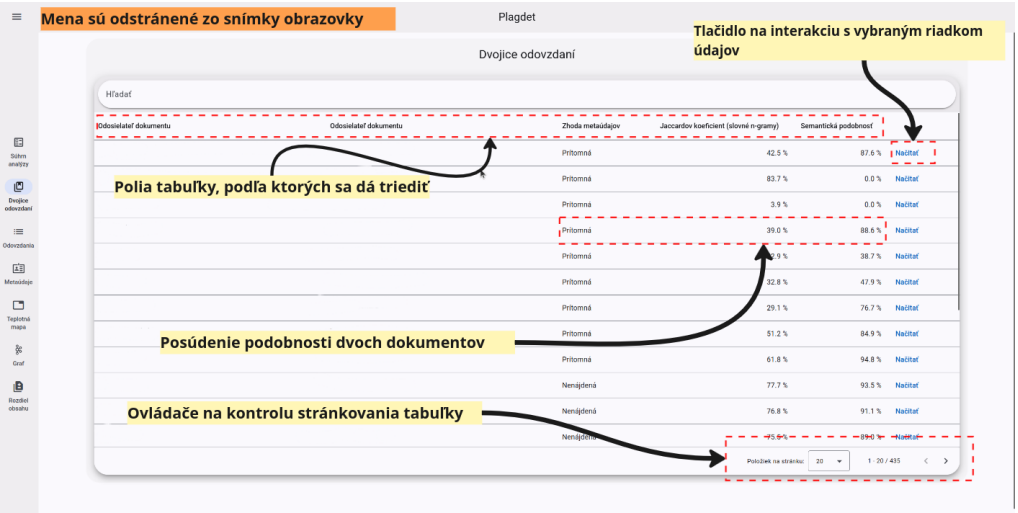
2.4 Práca s tabuľkami

Práca s tabuľkami je neoddeliteľná časťou pri hľadaní plagiátu, keďže v nich sú zahrnuté hlavné informácie o výsledkoch.

Všetky tabuľky sú vytvorené v rovnakom štýle a s rovnakým spôsobom interakcie, príklad je uvedený na obr. 2.3. Hranice tabuľky je možné odlíšiť ľahkým zvýšením plochy nad ostatnými prvkami stránky.

Hlavné komponenty tabuliek sú:

- vyhľadávacie pole – voliteľné, v závislosti od veľkosti tabuľky,
- ovládanie zobrazenej stránky údajov – voliteľné, v závislosti od veľkosti tabuľky,
- triediace záhlavia – všetky tabuľky podporujú triedenie podľa každého záhlavia,



Obrázok 2.3: Príklad tabuľky a zvýraznenie funkcií

- interakcia s celou tabuľkou – ak existuje, nachádza sa v pravom hornom rohu tabuľky,
- interakcia s riadkom tabuľky – voliteľné, ak je interakcia prítomná, bude pridané pole tabuľky s príslušným tlačidlom pre interakciu.

Vo veľkých tabuľkách je možné vykonávať vyhľadávanie podľa polí uvedených v tabuľke 2.1. Vo všetkých tabuľkách nie je dôležitá veľkosť písmen, diakritika však áno.

Kliknutím na záhlavia tabuľky je možné meniť triedenie tabuľky. Všetky tabuľky v programe podporujú triedenie podľa všetkých definovaných záhlaví.

Tabuľka 2.1: Tabuľky a vyhľadávacie polia

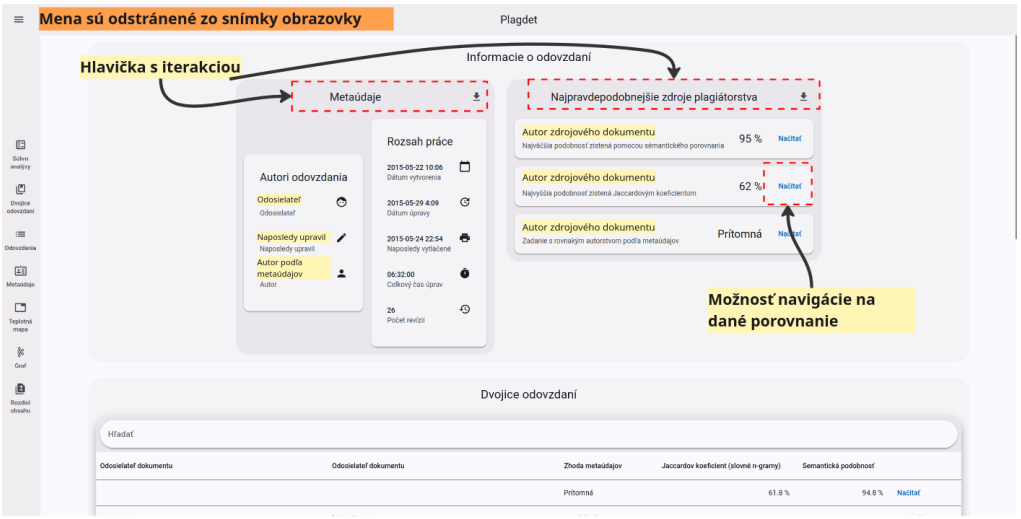
Tabuľka	Vyhľadávacie polia
Tabuľka dvojíc dokumentov	odosielateľ 1, odosielateľ 2
Tabuľka odovzdaní	odosielateľ
Tabuľka metaúdajov	odosielateľ, autor dokumentu, naposledy upravil

2.5 Stránka so zobrazením konkrétneho dokumentu

Táto stránka zodpovedá za zobrazenie informácií súvisiacich s hodnotením plagiátorstva pre určitý dokument.

Na tejto stránke sa nachádzajú:

1. zoznam všetkých uložených porovnaní daného dokumentu,



Obrázok 2.4: Obrázok stránky so zobrazením konkrétneho dokumentu

2. informácie prevzaté z metadát dokumentu,
3. pravdepodobné zdroje plagiátorstva,
4. obsah dokumentu zobrazený textovým editorom.

Pre metaúdaje odovzdania a pre zdroje plagiátov bola tiež vytvorená možnosť stiahnutia do formátu CSV. Polia, ktoré budú uložené pre metaúdaje, sú uvedené v tabuľke 2.2, a pre zdroje plagiátov v tabuľke 2.5. Tlačidlá na účel stiahnutia sa nachádzajú v hlavičkách komponentov, v pravom hornom rohu – príklad na obr. 2.4. Všetky komponenty, ktoré je možné stiahnuť, majú rovnakú farbu pozadia a rovnakú ikonu sťahovania. Práve tie údaje, ktoré sa nachádzajú v takýchto rámcoch, budú stiahnuté.

Na tejto stránke sa tiež nachádza veľký textový editor s obsahom danej práce. V ňom sa môžu vyskytovať farebné zvýraznenia textu s významom zodpovedajúcim legende v tabuľke 2.4.

Tabuľka 2.2: Exportované polia metaúdajov

Odosielateľ	Názov súboru	Cesta k súboru	Dátum vytvorenia
Naposledy upravil	Autor	Dátum úpravy	Čas úpravy
Počet revízií	Naposledy vytlačené		

Účel	Vzor farby
Zvýraznenie rovnakých hodnôt	
Zvýraznenie hodnôt, ktoré sa odchyľujú od priemeru	

Tabuľka 2.3: Legenda pre farby vyskytujúce sa v pri prehľade metaúdajov

Účel	Vzor farby
Zvýraznenie znenia zadania	
Zvýraznenie fragmentov kódu	
Zvýraznenie pravopisných chýb	
Zvýraznenie Unicode spoofingu	

Tabuľka 2.4: Legenda pre farby vyskytujúce sa v textovom editore

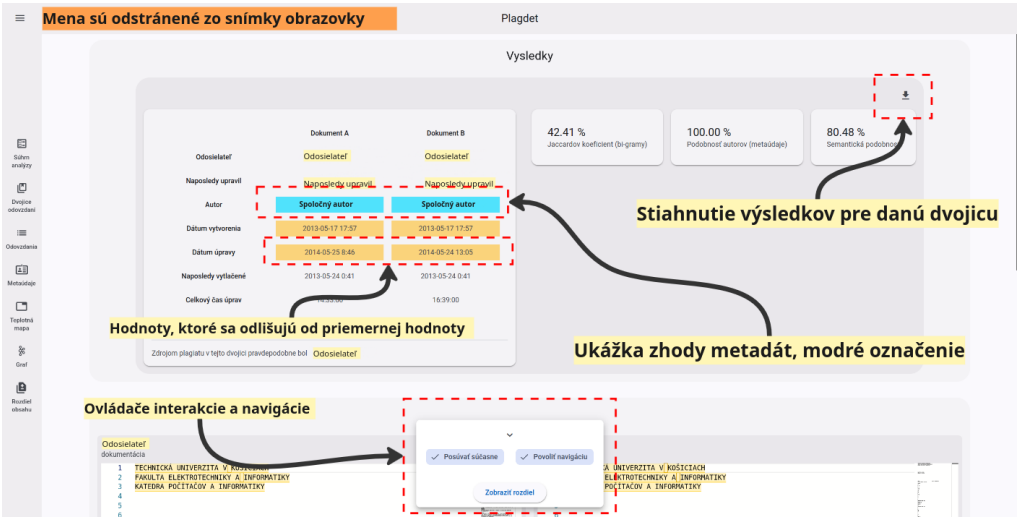
2.6 Stránka pre zobrazenie dvojice dokumentov

Na danej stránke vám bude predstavené porovnanie dvojice dokumentov. To zahŕňa porovnanie metaúdajov (obrázok 2.5), kde zvýraznenie polí je opísané v tabuľke 2.3. Porovnanie metaúdajov vyzerá ako mriežka na ľavej časti obrazovky, pričom výsledky – tri podpísané polia – sú zobrazené na pravej strane. Výsledky je možné stiahnuť aj do formátu CSV, tlačidlo sa nachádza vpravo hore. Polia, ktoré sa uložia, sú uvedené v tabuľke 2.5.

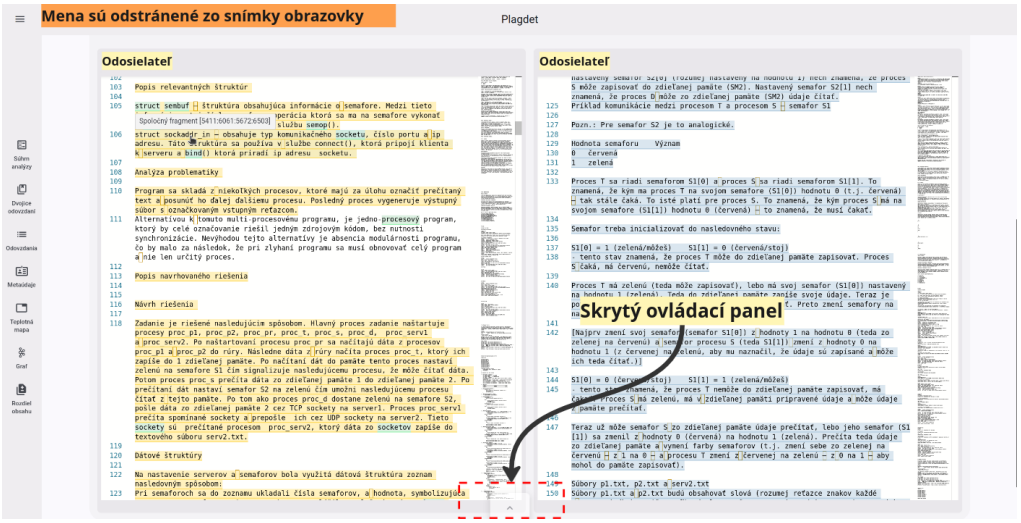
Pri porovnávaní obsahu dokumentu sa objaví jediné nové označenie s legendou v tabuľke 2.6. Každý takto zvýraznený fragment umožňuje navigáciu k zodpovedajúcemu úseku v druhom textovom editore. Túto možnosť navigácie je možné vypnúť v paneli v dolnej časti obrazovky, ktorý je zobrazený na obrázku 2.5. Editory taktiež podporujú synchronizované posúvanie, ktoré je možné vypnúť v tom istom paneli. Uvedený panel je možné skryť kliknutím na tlačidlo v tvare trojuholníka v jeho hornej časti (skrytý panel je zobrazený na obrázku 2.6).

Tabuľka 2.5: Exportované polia dvojíc dokumentov

Odosielateľ	Názov súboru	Cesta k súboru	Autor
Dátum vytvorenia	Naposledy upravil	Dátum úpravy	Čas úpravy
Počet revízií	Naposledy vytlačené	Zhoda metaúdajov	
Jaccardov koeficient (slovné n-gramy)		Semantická podobnosť	



Obrázok 2.5: Obrázok stránky dvojice dokumentov, prvá časť



Obrázok 2.6: Obrázok stránky dvojice dokumentov, druhá časť

Účel	Vzor farby
Zvýraznenie spoločných fragmentov	

Tabuľka 2.6: Legenda pre farby vyskytujúce sa v textovom editore pri prehľade dvojice dokumentov

2.7 Zobrazenie teplotnej mapy a rozdiel v obsahu

Na stránke s teplotnou mapou si môžete rýchlo prezrieť podobnosť každého dokumentu s každým v analyzovanej množine dokumentov (obrázok 2.7).

V riadkoch a stĺpcoch sú uvedení autori, pričom na ich priesečníkoch (bunkách matice) sa nachádza hodnota podobnosti. Typ zobrazenej podobnosti je možné meniť pomocou panela v dolnej časti stránky. Každá bunka mapy je klikateľná:

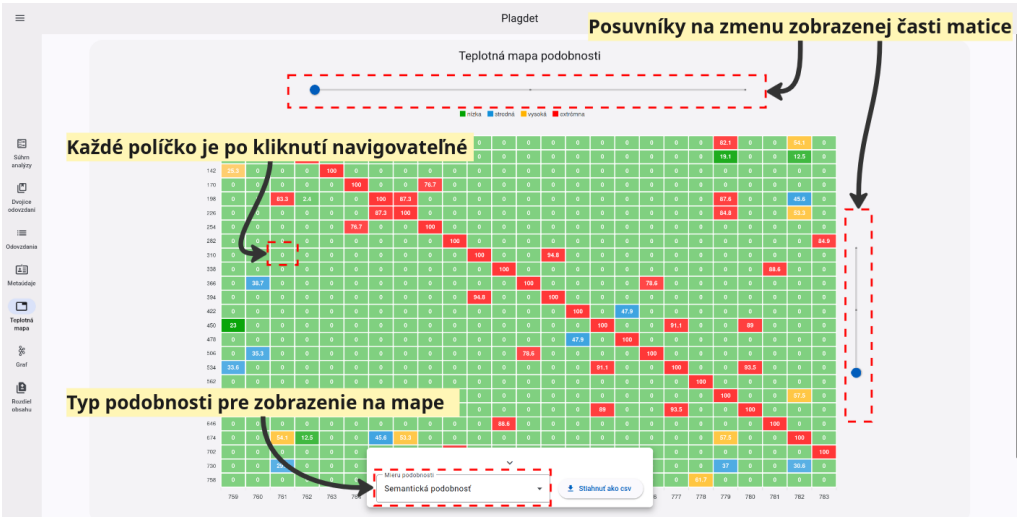
- ak kliknete na bunku, kde je autor rovnaký, otvorí sa daná práca,
- ak kliknete na bunku, pre ktorú existuje uložené porovnanie, otvorí sa dané porovnanie,
- ak kliknete na bunku, kde porovnanie dvojice dokumentov nie je uložené (napríklad neprekročilo prah podobnosti), otvorí sa porovnanie rozdielov obsahu.

Z dôvodu optimalizácie sa používa stránkovanie matice – na zmenu zobrazenej stránky použite vertikálne a horizontálne posuvníky (obrázok 2.7).

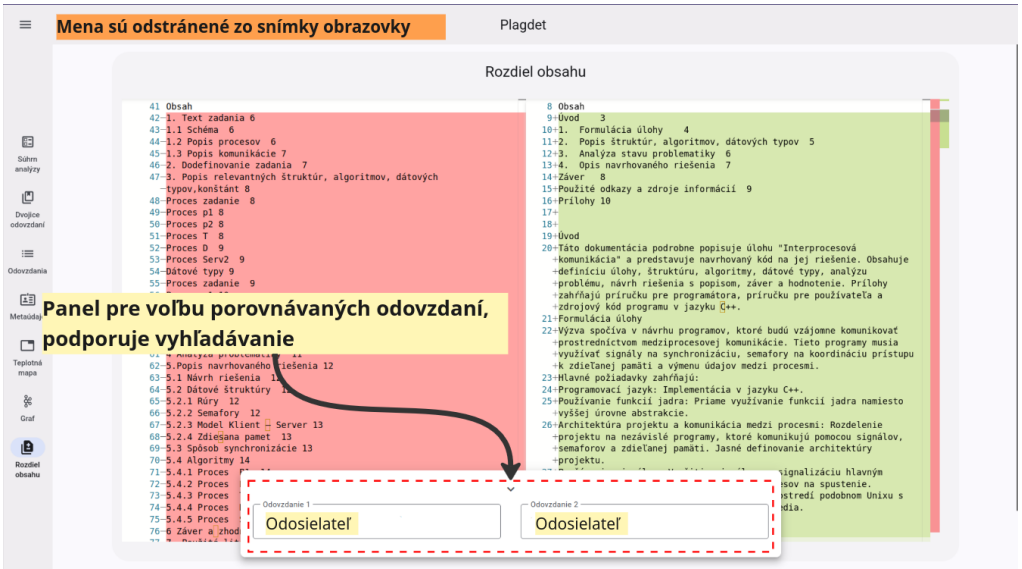
Na stránke „Rozdiel obsahu“ môžete porovnať dva dokumenty znak po znaku (princíp porovnávania podobný ako v Gite). Fragmenty zobrazené červenou farbou zodpovedajú častiam textu, ktoré boli odstránené (príklad označenia na obr. 2.8). Fragmenty zobrazené zelenou farbou sú tie, ktoré boli pridané. Časti bez zvýraznenia ukazujú na text, ktorý sa zhoduje.

2.8 Zobrazenie grafu odovzdaní

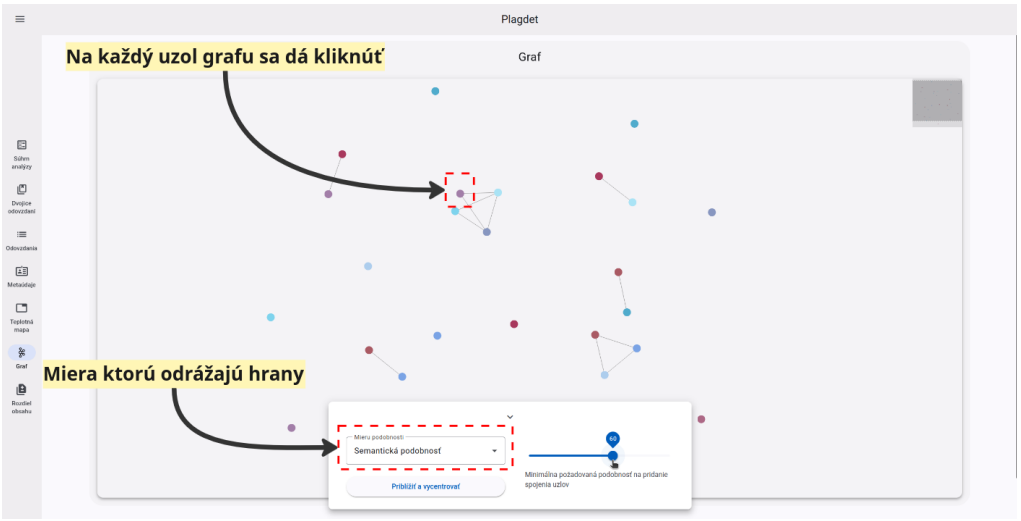
Stránka s grafom slúži na identifikáciu skupinového plagiátorstva. Vrcholy v grafe reprezentujú analyzované práce a hrany medzi nimi vyjadrujú podobnosť medzi týmito prácami (obrázok 2.9). Typ podobnosti, ktorý hrany zobrazujú, ako aj mieru podobnosti potrebnú na vytvorenie hrany, je možné zvoliť v paneli v dolnej časti stránky.



Obrázok 2.7: Obrázok stránky pre zobrazenie teplotnej mapy



Obrázok 2.8: Obrázok stránky pre zobrazenie rozdielu v obsahu



Obrázok 2.9: Obrázok stránky pre zobrazenie grafu odovzdaní

Na každý z vrcholov je možné kliknúť, čím sa zobrazí možnosť navigácie buď k príslušnej práci, alebo ku skupine prác, ku ktorej daná práca patrí.

Stránka so skupinou prác neprináša nové funkcionality, ale zobrazuje filtrované údaje, ktoré sú relevantné pre dané zadania.

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Softvér na identifikáciu plagiátov technickej dokumentácie

Príloha C: Systémová príručka

Bakalárska práca

2025 **Taras Hamkalo**

Obsah

1 Dokumentácia programu na porovnanie dokumentov **1**

1.1 Prehľad štruktúry 1

1.2 Hlbší prehľad modulov, „plagdet-core“ 2

1.2.1 Balík `*.documents` 3

1.2.2 Balík `*.io` 4

1.2.3 Balík `*.preprocessing` 6

1.2.4 Balík `*.indexing` 6

1.2.5 Balík `*.classification` 7

1.2.6 Balík `*.comparison` 8

1.2.7 Hlavná trieda Plagdet a externé závislosti 9

1.3 Hlbší prehľad modulov, „plagdet-cli“ 10

1.4 Konkrétnejší opis vybraných balíkov a tried 10

1.4.1 Predspracovanie textu 10

1.4.2 Trieda `CodeSegmentAnnotator` 11

1.4.3 Trieda `TextBlockIdentifierAnnotator` 11

1.4.4 Trieda `TextBlockSentenceSplitter` 11

1.4.5 Trieda `MorphoDiTaAnalysisAnnotator` 11

1.4.6 Trieda `WordFilteringAnnotator` 12

1.4.7 Trieda `SpellingAnnotator` 12

1.4.8 Trieda `TemplateSentencesAnnotator` 12

1.5 JAR zdroje a konštanty 12

2 Dokumentácia pre používateľské rozhranie **14**

2.1 Inštalácia závislosti pre začiatok práce 14

2.2 Prehľad štruktúry 14

2.3 Hlbší prehľad projektu, rozdelenie závislostí a komponentov 15

2.4 Opis stavu aplikácie a načítanie údajov 16

2.5 Opätovné využitie stránok 16

2.6 Opis vybraných balíkov a tried 17

Zoznam obrázkov

1.1 Diagram tried pre model reprezentácie dokumentov 3
1.2 Diagram tried pre triedy typu pool 4
1.3 Diagram tried pre iteráciu korpusom 4
1.4 Diagram tried pre čitačky dokumentov 5
1.5 Diagram tried pre iteratory používateľským vstup 5
1.6 Diagram tried, pre hlavné triedy balíka `*.preprocessing.lucene` 7

Zoznam tabuliek

1.1 Zoznam Lucene polí so zodpovedajúcimi typmi a účelom 8
1.2 Prehľad externých závislostí použitých v „plagdet-core“ 9
1.3 Prehľad externých závislostí použitých v „plagdet-cli“ 10
1.4 Názvy zdrojov a tried, ktoré od nich závisia 13

1 Dokumentácia programu na porovnanie dokumentov

Program na porovnávanie dokumentov je napísaný v jazyku Java, jazyková úroveň 17, odporúčaná distribúcia je Eclipse Temurin.

Na zostavenie projektu sa používa nástroj Gradle vo verzii 8.10.1, v Groovy DSL verzii.

Na inštaláciu projekt naklonujte repozitár z Gitu pomocou nasledujúceho príkazu [1]:

```
git clone {repository-url}
```

Odporúčané vývojové prostredie pre projekt je JetBrains IntelliJ IDEA, pre ktoré bola pridaná predbežná konfigurácia projektu.

Na overenie inštalácie pridajte do zoznamu spustiteľných úloh v IntelliJ IDEA úlohu `:plagdet-cli:run` a spustite ju. Po úspešnom dokončení sa v konzole zobrazí výpis argumentov programu.

1.1 Prehľad štruktúry

Program je rozdelený do dvoch hlavných Gradle modulov:

- plagdet-core,
- plagdet-cli.

„plagdet-core“ zodpovedá za celú logiku porovnávania, teda načítanie, spracovanie a nakoniec porovnanie dokumentov. Modul „plagdet-core“ zostáva úplne nezavislý – všetky nadstavby (napr. vytvorenie serverovej verzie, pridanie Springu) by nemali ovplyvniť tento modul. Modul by nemal byť aplikáciou, ale slúžiť ako knižnica pre ostatné, pričom je východiskovým bodom celého systému.

[1] Ako `repository-url` použitie odkaz `git@git.kpi.fei.tuke.sk:kpi-zp/2025/bp.taras.hamkalo/plagiarism-detection.git`

„plagdet-cli“ je nadstavbou nad „plagdet-core“, pričom pridáva logiku pre zhromažďovanie parametrov z terminálu. Obsahuje triedu `Main`, ktorá momentálne predstavuje hlavnú vstupnú triedu aplikácie. Použite túto triedu na spúšťanie debug relácií a vytváranie spustiteľných JAR súborov.

Nastavenie systému zostavenia Gradle je rozdelené medzi nasledujúce súbory (všetky cesty sú od koreňa projektu):

- /build.gradle – definuje spoločné vlastnosti a závislosti pre všetky projekty a podprojekty. Napríklad výber logovacieho systému alebo testovacieho frameworku.
- /settings.gradle – definuje globálne vlastnosti projektu, ako napríklad, ktoré podmoduly zahrnúť do zostavy.
- .gradle.properties – definuje premenné, ktoré je možné používať vo všetkých podprojektoch. Obsahuje všetky verzie knižníc používaných v programe.
- /plagdet-core/build.gradle – definuje závislosti a pluginy pre modul „plagdet-core“.
- /plagdet-cli/build.gradle – definuje závislosti a pluginy pre modul „plagdet-cli“.

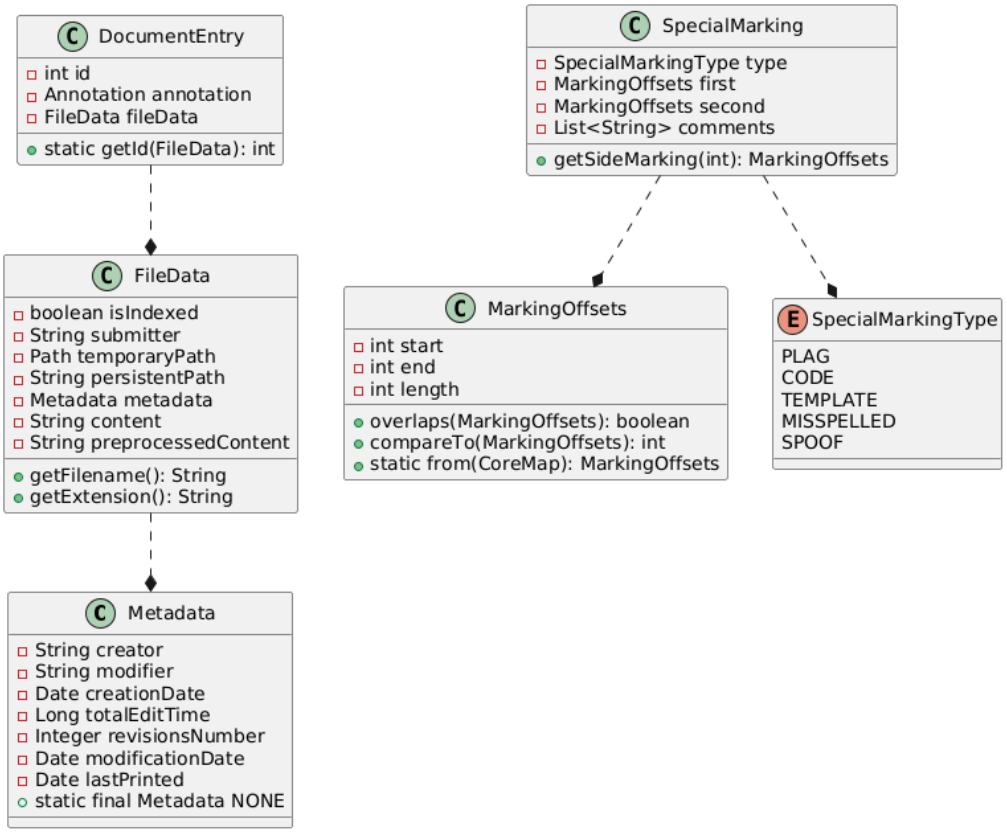
Ak význam niektorého z týchto súborov nie je jasný, odporúčam bližšie sa oboznámiť s projektom Gradle.

Dôležitým adresárom je „libs“ – momentálne sa v ňom nachádzajú artefakty projektu MorphoDiTa, konkrétne natívna knižnica (SO) a jej JAR. Pre pridávanie ďalších natívnych závislostí ich ukladajte do tohto adresára. Projekt Gradle je nastavený tak, aby tento adresár rozpoznal ako repozitár závislostí.

1.2 Hlbší prehľad modulov, „plagdet-core“

Začnime modulom „plagdet-core“. Ako už bolo spomenuté, tento modul zodpovedá za implementáciu celej logiky porovnávania [2]. V tejto sekcii sa budú priblížené niektoré top-level balíky programu.

[2] Spoločný prefix všetkých balíkov v tomto module je `dev.taras.hamkalo.plagdet`



Obrázok 1.1: Diagram tried pre model reprezentácie dokumentov

1.2.1 Balík `*.documents`

Tento balík definuje základný model reprezentácie dokumentov v programe a iterátory nad kolekciou dokumentov.

Model je reprezentovaný triedami

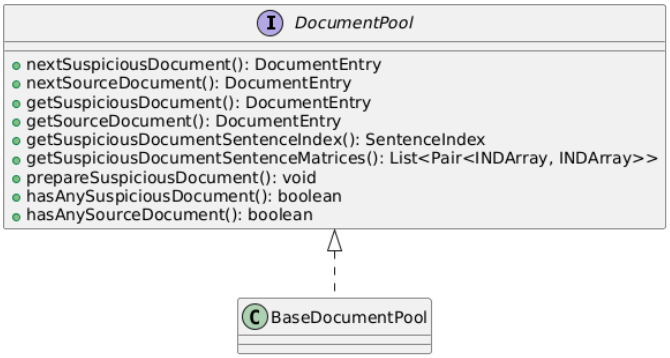
- `DocumentEntry`,
- `FileData`,
- `Metadata`,
- `SpecialMarkings`

.

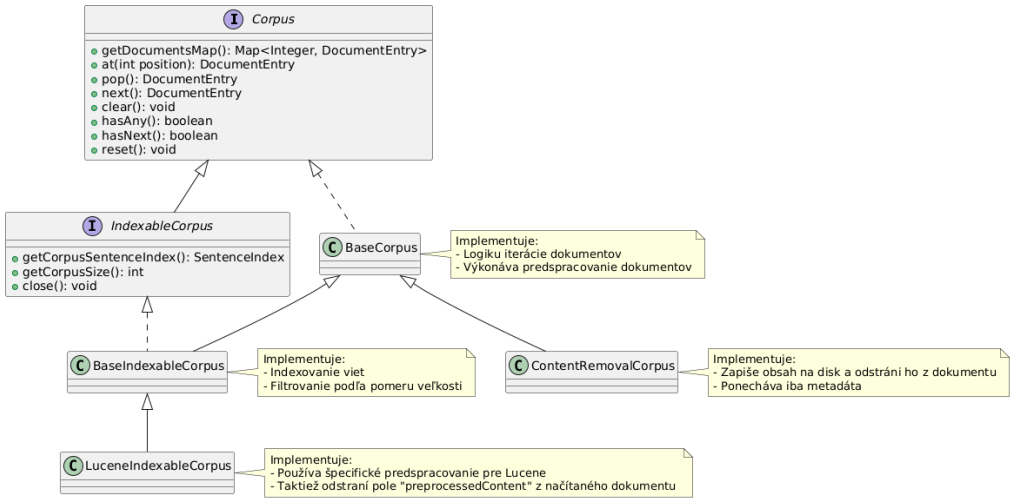
Diagram týchto tried je s uvedený na obrázku 1.1. Ich použitie bude jasnejšie po prečítaní opisov nasledujúcich balíkov.

Iterátory nad kolekciou dokumentov v tomto balíku sú rozdelené do dvoch typov: `Corpus` a `Pool`.

`Corpus` iterátory (diagram tried na obrázku 1.3) sa používajú na transparentnú iteráciu nad načítanými a predspracovanými dokumentmi. Ich cieľom je opti-



Obrázok 1.2: Diagram tried pre triedy typu pool



Obrázok 1.3: Diagram tried pre iteráciu korpusom

malizovať túto iteráciu oneskoreným načítavaním dokumentov a napríklad zapisovaním obsahu dokumentu na disk, aby sa šetrila pamäť.

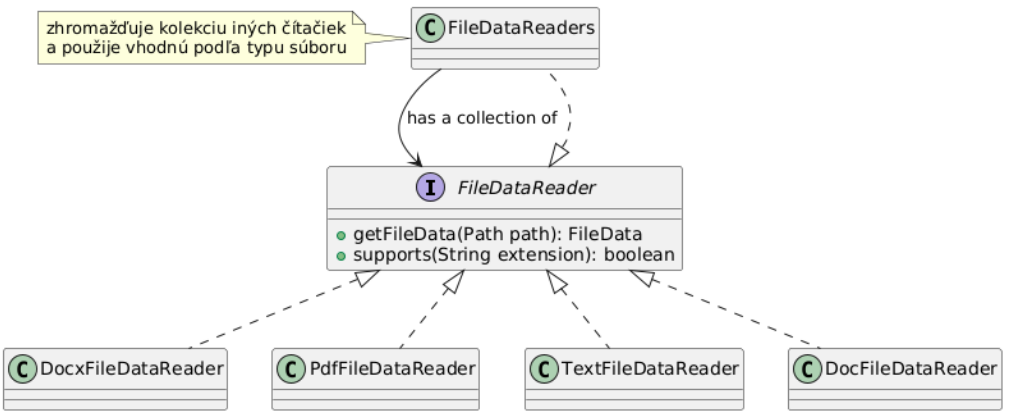
`Pool` iterátory (diagram 1.2) sú objekty s množstvom závislostí, ktoré budú vysvetlené v ďalších balíkoch. Používajú sa na efektívnejšie využívanie zdrojov koordinovaním ich alokácie (napr. jednorazové vytváranie indexu viet pre podozrivý dokument). Zároveň poskytujú intuitívnejšie metódy iterácie pri vnútrokorpusovom odhaľovaní plagiátov, ako napríklad `nextSuspiciousDocument()` alebo `nextSourceDocument()`.

1.2.2 Balík `*.io`

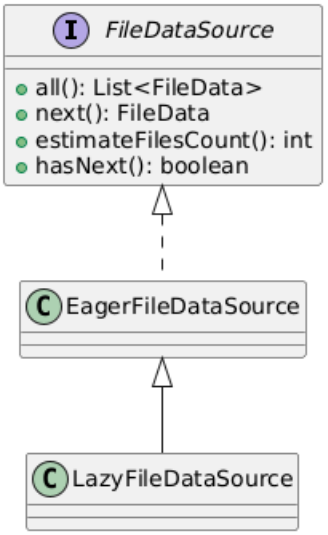
Tento balík zodpovedá za celú logiku získavania dát zo vstupu od používateľa a následného zápisu výsledkov na disk.

V tomto balíku sú definované pomocné triedy:

- `IOUtils` – práca s Java zdrojmi (napr. kopírovanie do dočasného súboru),



Obrázok 1.4: Diagram tried pre čitačky dokumentov



Obrázok 1.5: Diagram tried pre iteratory používateľským vstup

- `ZipArchiver` – rekurzívne prehľadanie adresára a vytvorenie jeho ZIP archívu,
- `ArchiveExtractor` – extrakcia ZIP súborov, s možnosťou voľby formátov pre extrahované položký.

Tento balík využíva model z balíka `*.documents` a napĺňa ho reálnymi dátami. Na tento účel je v ňom pre každý typ dokumentu definovaný vlastný čítač. Implementované čítače sú ukázané na diagrame tried, obr. 1.4.

V tomto balíku sú tiež definované iterátory pre používateľský vstup – diagram tried je na obrázku 1.5.

Pri pridávaní nového formátu, implementujte rozhranie `FileDataReader`, pridajte nový formát do podporovaných formátov v `EagerFileDataSource` a vytvorte objekt vašej čítačky v triede `Plagdet`.

Súčasťou tohto balíka je logika zápisu výsledkov na disk. V tomto baliku je definovany model serializacie objektov pomocou knižnice Jackson (formát JSON). Všetky rozšírenia modelu výsledkov implementujte v tomto balíku, pridaním tried definujuich model a rozsirenim `FileReportWriter`.

Trieda `BufferedReportWriter` preberá zodpovednosť za zníženie počtu zápisov na disk. Kľúčové aspekty sú:

- bufferovanie výsledkov porovnávania,
- deduplikácia záznamov (napr. ak majú dva dokumenty referenciu na rovnaké porovnanie, nemal by sa tento záznam zapísať dvakrát).

1.2.3 Balík `*.preprocessing`

Tento balík zodpovedá za celú logiku predspracovania a je implementovaný na základe Stanford Core NLP.

Trieda `PlagdetAnnotations` definuje všetky vlastné anotácie, ktoré sa pridávajú k dokumentom. Vystupuje ako enumerácia (enum) typov anotácií v programe. Príkladmi anotácií sú znakové offsety označujúce kód alebo zadanie, pravopisné chyby v texte a podobne. Pre podrobnejšie zoznámenie odporúčam prezrieť túto triedu – ku každej anotácii je pridaný komentár s opisom použitia a samotné anotácie sú samovysvetľujúce.

Podbalík `annotators` definuje všetky triedy pre predspracovanie textu, implementujúce rozhranie Stanford Annotator.

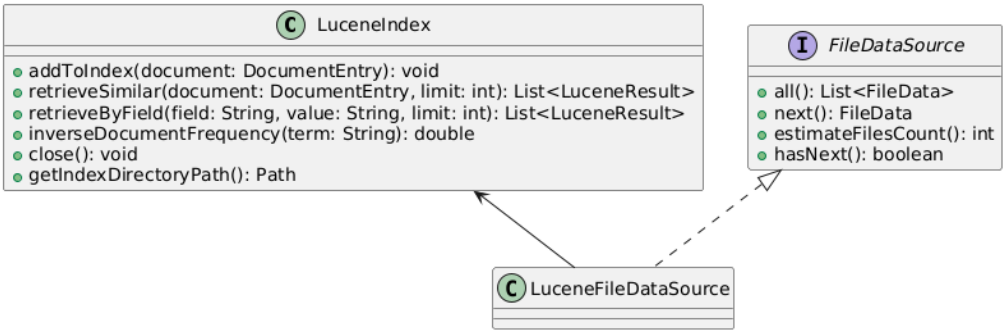
V tomto balíku sú definované aj dve pomocné triedy:

- `AnnotationSerializer` – serializácia a deserializácia Stanford Annotation, podľa gramatiky definovanej v hlavnej práci (časť „Indexovanie a získanie kandidátov“),
- `AnnotationUtils` – metódy, prostredníctvom ktorých triedy mimo tohto balíka používajú anotácie Stanfordu.

1.2.4 Balík `*.indexing`

V tomto balíku je definované ako indexovanie dokumentov (full-textové vyhľadávanie), tak aj indexovanie viet dokumentu.

Implementácia indexovania dokumentov je realizovaná pomocou knižnice Apache Lucene.



Obrázok 1.6: Diagram tried, pre hlavné triedy balíka `*.preprocessing.lucene`

Triedou, ktorá implementuje logiku indexovania, je `LuceneIndex`. Ona definuje metódy na pridanie dokumentu a jeho vyhľadávanie podľa metadát a obsahu (diagram tried 1.6).

Na rozšírenie polí dokumentu, ktoré sa ukladajú do indexu, použite triedy z podbaliku `mapping`:

- `FileDataMapper`,
- `MetadataMapper`,
- `StanfordAnnotationMapper`.

Všetky aktuálne ukladané polia sú uvedené v tabuľke 1.1.

V tomto balíku je tiež rozšírené rozhranie `FileDataSource`. Tato implementácia zodpovedá za zabezpečenie transparentnej integrácie Lucene do procesu porovnávania.

Indexovanie viet je implementované pomocou knižnice `Jvector`. Jej implementácia je pomerne priamočiara a bola vytvorená podľa príkladov uvedených v ich dokumentácii [3].

1.2.5 Balík `*.classification`

Tento balík definuje triedy pre klasifikátory používané aplikáciou. V súčasnosti obsahuje len metódu trénovania klasifikátora a extrakciu vlastností potrebných na klasifikáciu kódu.

[3] `https://github.com/datastax/jvector`

Názov poľa	Typ poľa	Účel
`id`	`StringField` (Store.NO)	Identifikátor dokumentu, vytvorený na základe názvu adresára, v ktorom sa nachádza, a názvu dokumentu
`path`	`StringField` (Store.YES)	Názov posledného adresára na ceste k súboru, a názov dokumentu.
`content`	`TextField` (Store.YES)	Pôvodný obsah súboru ako text.
`preprocessed_content`	`TextField` (Store.YES)	Predspracovaný obsah dokumentu, uložený pre deserializáciu Stanford Annotation.
`search_content`	`TextField` (Store.NO)	Spracovaný obsah dokumentu vhodný na vyhľadávanie.
`submitter`	`StringField` (Store.YES)	Meno osoby, ktorá dokument odovzdala.
`creator`	`StringField` (Store.YES)	Osoba, ktorá vytvorila dokument podľa metaúdajov.
`creation_date`	`StringField` (Store.YES)	Dátum vytvorenia dokumentu.
`total_edit_time`	`StringField` (Store.YES)	Celkový čas úpravy dokumentu.
`modifier`	`StringField` (Store.YES)	Osoba, ktorá naposledy upravila dokument.
`modification_date`	`StringField` (Store.YES)	Dátum poslednej úpravy dokumentu.
`last_printed`	`StringField` (Store.YES)	Dátum posledného vytlačenia dokumentu.
`revisions_number`	`StringField` (Store.YES)	Počet revízií dokumentu.

Tabuľka 1.1: Zoznam Lucene polí so zodpovedajúcimi typmi a účelom

1.2.6 Balík `*.comparison`

Tento balík je zodpovedný za logiku porovnávania nezávislú od zdroja alebo formátu dokumentu.

Daný balík obsahuje tieto podbalíky:

- `*.comparison.model` – definovanie všetkých objektov na výpočet a uloženie výsledkov porovnania,
- `*.comparison.filter` – definovanie metód pre rozhodnutie, či je potrebné pokračovať v porovnaní niektorej dvojice dokumentov,
- `*.comparison.util` – triedy pre výpočet vzdialeností medzi zhodami v dokumente a metódy výpočtu miery plagiátorstva,
- `*.comparison.scorer` – triedy pre výpočet hodnoty podobnosti medzi vetami,
- `*.comparison.blocks` – definície interfejsov `SeedingStrategy`, `ExtensionStrateg` `FilteringStrategy` a ich implementácie.

Skupina	Artefakt	Verzia
org.apache.lucene	lucene-core, lucene-queryparser, lucene-analyzers-common, lucene-backward-codecs, lucene-codecs	8.10.1
org.apache.poi	poi-ooxml, poi-scratchpad	5.4.0
org.apache.commons	commons-compress	1.27.1
org.apache.commons	commons-text	1.13.0
com.itextpdf	itext7-core	7.2.5
io.github.jbellis	jvector	3.0.6
:morphodita:	–	–
edu.stanford.nlp	stanford-corenlp	4.5.8
org.deeplearning4j	deeplearning4j-nlp	1.0.0-M2.1
org.nd4j	nd4j-native	1.0.0-M2.1
org.bytedeco	openblas (classifier: linux--x86_64)	0.3.13-1.5.7
org.bytedeco	javacpp	1.5.7
com.fasterxml.jackson.core	jackson-databind, jackson-core, jackson-annotations	2.18.3
org.languagetool	language-sk	6.6.12-3
com.ibm.icu	icu4j	76.1

Tabuľka 1.2: Prehľad externých závislostí použitých v „plagdet-core“

Najdôležitejšie triedy tohto balíka sú:

- Comparator – zhromažďuje všetky vyššie uvedené komponenty a volá ich v správnom poradí,
- Embeddings – mapuje vety a slová na vektory podľa Word2Vec.

1.2.7 Hlavná trieda Plagdet a externé závislosti

Plagdet je miesto v aplikácii, kde sa vykonáva vytváranie všetkých objektov a kontrola priebehu porovnávania. Túto triedu by mali používať rozširujúce moduly, ak chcú porovnávať alebo indexovať dokumenty.

Aplikácia používa rad externých knižníc, ktoré sú zhrnuté v tabuľke 1.2.

Skupina	Artefakt	Verzia
commons-cli	commons-cli	1.9.0

Tabuľka 1.3: Prehľad externých závislostí použitých v „plagdet-cli“

1.3 Hlbší prehľad modulov, „plagdet-cli“

Tento modul, ako už bolo spomenuté, obsahuje logiku na zber parametrov pre spustenie programu z terminálu. Obsahuje tieto triedy:

- Main – trieda štartér,
- OptionsBuilder – trieda, ktorá implementuje návrhový vzor Builder a zjednodušuje registráciu dostupných parametrov v aplikácii,
- OptionsMapper – trieda, ktorá mapuje argumenty programu (ich textovú formu) na objekty, ktoré sa odovzdávajú triede `Plagdet`,
- PlagdetCli – trieda v ktorá registruje parametre, mapuje ich a volá príslušné metódy z triedy `Plagdet`.

Daný modul má len jednu externú závislosť uvedenú v tabuľke 1.3.

1.4 Konkrétnejší opis vybraných balíkov a tried

Táto časť objasňuje niektoré implementačné detaily, ktoré boli v predchádzajúcich častiach vynechané.

1.4.1 Predspracovanie textu

Hlavnou triedou, v ktorej sa vykonáva predspracovanie, je `*.preprocessing.Pipeline`. Vytvára objekty Stanford Pipeline v závislosti od konfigurácie používateľa a opätovne ich používa pre každý nový dokument. Sú v nej vytvorené 3 potrubia:

- firstStagePipeline – na účely prvotného predspracovania textu (kroky sú opísané v hlavnej časti práce),
- templateComparisonPipeline – vykonáva sa po firstStagePipeline a má za cieľ odstrániť všetky zhody vstupného dokumentu so zadanou šablónou (napr. text úlohy),
- luceneDocumentPipeline – definuje spôsob predspracovania dokumentu získaného z indexu Lucene (napr. vykoná deserializáciu).

1.4.2 Trieda `CodeSegmentAnnotator`

Účelom tejto triedy je detekovať kód a odstrániť ho. Zároveň dodá metadáta o umiestnení kódu.

Na dokument umiestňuje nasledujúce anotácie:

- TextBlockBoundaries – táto anotácia obsahuje index poslednej vety pred začiatkom fragmentu kódu. Používa sa na identifikáciu hraníc textových odsekov,
- CodeSegmentAnnotation – ukladá maximálne po sebe nasledujúce fragmenty kódu, konkrétne ich znakové posuny. Dané posuny sú potrebné iba na zobrazenie výsledkov.

1.4.3 Trieda `TextBlockIdentifierAnnotator`

Účelom tejto triedy je priradiť identifikátor textového bloku každej vete dokumentu. Funguje na základe anotácie TextBlockBoundaries a všetkým vetám v tom istom bloku priraďuje rovnaké ID.

Poskytuje nasledujúcu anotáciu:

- TextBlockId – anotácia obsahujúca jedno číslo zodpovedajúce ID bloku. Nachádza sa len vo vetách dokumentu.

Táto trieda by sa mala volať v potrubí po každej zmene hraníc textového bloku. Bloky sa prerozdelia vzhľadom na existujúce ID a nové hranice. Po vykonaní sa anotácia TextBlockBoundaries z dokumentu odstráni.

1.4.4 Trieda `TextBlockSentenceSplitter`

Účelom tohto anotátora je prerozdeliť tokeny z viet v tom istom bloku (vety s rovnakou hodnotou anotácie TextBlockId). Robí to na základe interpunkčných znamienok a neskôr sa pokúša zvážiť dĺžku viet ich prerozdelením do vedľajších viet.

1.4.5 Trieda `MorphoDiTaAnalysisAnnotator`

Tento anotátor funguje ako lematizátor a tagger využívajúci MorphoDiTa. Pridáva nasledujúcu vlastnú anotáciu:

- GuessedLemmaAnnotation – ukazuje, či anotátor rozpoznal slovo.

1.4.6 Trieda `WordFilteringAnnotator`

Účelom tohto anotátora je filtrovanie tokenov a viet v dokumente. Odstraňuje z viet a dokumentu všetky neslovné tokeny podľa regexu a dĺžky (menej ako 3 písmená). Filtruje aj vety dokumentu – odstránia sa všetky vety s menej ako 2 tokenmi.

Táto trieda tiež kontroluje, či slovo nie je Unicode spoof. Ak ním slovo je, anotátor ho nahradí jeho unicode kostrou (viac v dokumentácii Unicode). K takýmto slovám sa pridá anotácia `SpoofedWordAnnotation`.

1.4.7 Trieda `SpellingAnnotator`

Tento anotátor sa používa na zvýšenie presnosti lematizácie a vyhľadávania neurónových vektorových reprezentácií. Spôsob výberu korekcie je podrobne opísaný v hlavnej práci.

Oprava slov sa vykonáva **iba** pre slová, ktoré majú nastavenú anotáciu `GuessedLemmaAnno`
pomocou anotátora `MorphoDiTaAnalysisAnnotator`. To pomáha výrazne urýchliť krok opravy pravopisu, pretože pri niektorých slovách ich už morfologický analyzátor skontroloval.

Po tomto kroku sa opäť spustí morfologický analyzátor, ktorý by už mal rozpoznať opravené slovo.

1.4.8 Trieda `TemplateSentencesAnnotator`

Tento anotátor sa používa na porovnávanie a odstraňovanie fragmentov textu šablóny, ako je napríklad text úlohy.

Pridáva tieto dve anotácie:

- TemplateSentencesAnnotation – ukladá ikonické posuny začiatku a konca šablónových viet,
- TextBlockBoundaries – táto anotácia obsahuje index poslednej vety pred začiatkom šablóny.

1.5 JAR zdroje a konštanty

V aplikácii je pomerne veľa zdrojov, ktoré by mali byť dodané pri spustení. Názvy týchto zdrojov a to, ktorá trieda od nich závisí, sú zdokumentované v tabuľke 1.4.

Názov triedy	**Zdroj**	**Opis**
Embeddings	models/sk.poslemma.prim-10.0-public-saneword2vec.bin	POS a lemmatizačný model typu word2vec
CodeSegmentAnnotator	models/code-classifier.bin	Klasifikačný model na detekciu a anotáciu úsekov kódu v texte
MorphoDiTaAnalysisAnnotator	models/slovak-morfflex-pdt-170914-pos_only.tagger	Model pre POS tagovanie a lemmatizáciu v MorphoDiTa
StopWordAnnotator	sk-stopwords.txt	Zoznam slovenských stopslov
TemplateSentencesAnnotator	template.txt	Šablónový text, vylúčený z analýzy, predvolený pre úlohu IPC

Tabuľka 1.4: Názvy zdrojov a tried, ktoré od nich závisia

2 Dokumentácia pre používateľské rozhranie

V danej kapitole je priblížená metoda implementácie používateľského rozhrania.

2.1 Inštalácia závislosti pre začiatok práce

Program na vizualizáciu výsledkov analýzy je napísaný pomocou rámca Angular vo verzii 19.0.2. Pre začiatok práce je potrebné nainštalovať distribúciu Node.js – riaďte sa pokynmi na ich webovej stránke. Nainštalujte si tiež rozšírenie angular cli, pretože príkazy definované v súbore package.json sú zapísané skrátenou syntaxou, ktorá nebude fungovať bez tohto rozšírenia.

S ohľadom na to vykonajte nasledujúce kroky:

```
1. git clone {repository-url}
2. npm install
3. npm install -g @angular-cli
```

Odporúčané vývojové prostredie pre projekt je JetBrains WebStorm. Pre overenie správnosti inštalácie vykonajte nasledujúci príkaz:

```
ng serve
```

Po spustení by mala byť aplikácia dostupná na adrese `http://localhost:4200`.

2.2 Prehľad štruktúry

Program je implementovaný ako jeden modul so štandardnou štruktúrou pre Angular. Dôležité súbory a adresáre sú:

- `angular.json` – definícia toho, ako vykonávať hlavné Angular príkazy ako `build` alebo `serve`

- `tsconfig.json` – definuje úroveň jazyka TypeScript
- `eslint.config.js` – definícia štýlovacích pravidiel používaných nástrojom ESLint
- `package.json` – definícia všetkých závislostí a vlastných príkazov
- `src/assets/` – adresár obsahujúci verejné zdroje ako ikony a texty uložené vo formáte JSON
- `src/app` – adresár obsahujúci všetky implementované závislosti

2.3 Hlbší prehľad projektu, rozdelenie závislostí a komponentov

Implementácia funkcií systému je rozdelená podľa rolí v Angulari, ktoré vykonávajú zodpovedajúce triedy. Napríklad:

- `app/directives` – implementované Angular direktívy,
- `app/pipes` – implementované `pipe` pre spracovanie textu v HTML šablónach,
- `app/services` – implementované Angular služby,
- `app/guards` – implementované Angular `guards`.

Ďalšie rozdelenie v projekte je podľa rolí v systéme a typu komponentu. Príklady:

- `app/pages` – obsahuje kľúčové stránky, ktoré nemajú vlastnú logiku, ale slúžia na zoskupenie viacerých komponentov,
- `app/components` – obsahuje všetky komponenty aplikácie, zvyčajne s logikou iba jednej funkcie,
- `app/components/cards` – všetky Angular Material karty,
- `app/components/base` – komponenty bez vlastnej špecifickej úlohy, ktoré však poskytujú základ pre iné funkcie,
- `app/components/tables` – všetko, čo súvisí s tabuľkami.

V projekte je tiež oddelený dátový model a typy údajov, ktoré používajú služby. Podstata rozdelenia:

- `app/model` – kľúčové štruktúry na zobrazenie dátového modelu aplikácie, napr. porovnanie dvojice dokumentov alebo odovzdanie jedného autora,
- `app/types` – pomocné štruktúry, bez ktorých by bola ďalšia manipulácia s dátami neefektívna, napr. `file-wrapper`, `nav-item`. uzol grafu a iné

Štruktúra stránky je definovaná koreňovým komponentom `app/layout`. Nachádzajú sa v ňom spoločná hlavička, panel pre navigáciu a `router` pre zobrazenia stránok.

Registrácia nových stránok prebieha v konfiguračnom súbore `app/app.routes.ts`. Na rozšírenie využite príklad uvedený v tomto súbore. Pre registráciu novej stránky do navigačného panela je potrebné rozšíriť zoznam stránok v navigačnej službe `app/services/navigation/navigation.service.ts`. Na základe príkladu v tomto súbore definujte novú stránku, jej opis v navigácii, ikonu a zoznam navigácií, ktoré sa zobrazia, keď na túto stránku používateľ prejde. Rozšírenie ikon vykonajte pridaním do zoznamu ikon v triede `app/icons-registratoin.service.ts` podľa príkladu uvedeného v tomto súbore. Ikonu umiestnite do adresára `app/assets`.

2.4 Opis stavu aplikácie a načítanie údajov

Najzložitejšou časťou pri rozširovaní aplikácie môže byť identifikácia údajov, s ktorými chcete pracovať.

Po nahratí ZIP archívu analýzy používateľom začína jeho rozbaľovanie a vytváranie tabuľky prác a dvojíc dokumentov s kľúčmi zodpovedajúcimi ich ID. Túto funkcionalitu implementuje služba `app/services/file-utils.service.ts`. Pri zmene formátu údajov upravte logiku načítania v tomto súbore.

Po načítaní všetkých údajov sa vytvorí objekt typu `app/services/model/report.ts`, s ktorým aplikácia pracuje počas celej doby. Aktuálna verzia tohto objektu je dostupná v kontexte aplikácie `app/context/analysis-context.service.ts`. Nastavenie tohto kontextu prebieha na úvodnej stránke:

`app/pages/analysis/analysis-page`.

2.5 Opätovné využitie stránok

Aby sa vaša stránka nekreslila sa opakovane, v aplikácii je implementovaná Angular `RouteReuseStrategy` v triede:

`app/context/plagdet-route-reuse-strategy.ts`.

Podľa príkladu uvedeného v tomto súbore pridajte vašu požadovanú stránku.

Známy problém pri tomto type opätovného využitia komponentov sa týka knižnice ApexCharts. Jej komponenty sa nepodarilo efektívne znovu využiť, keďže dochádzalo k strate údajov.

Pre jednoduchšie uchovávanie dát – napríklad ak je potrebné komponent prekresliť, ale chcete zachovať výsledky náročného výpočtu použite:

`app/context/route-context.service.ts`.

Ide o statickú službu na ukladanie párov kľúč–hodnota.

Dôležité je, že akýkoľvek kontext alebo cache sa vymaže pri návrate na domovskú stránku. Túto stránku môžete zmeniť v:

`app/services/navigation/navigation.service.ts`.

2.6 Opis vybraných balíkov a tried

V aplikácii sa nenachádza príliš zložitá logika spracovania – vo väčšine prípadov ide len o zobrazenie údajov na stránke.

Ak potrebujete pridať novú kartu (card), pridajte ju do `app/components/cards`, pričom využite `app/components/cards/shared`, aby ste zachovali jednotný štýl.

Ak potrebujete pridať novú tabuľku, pridajte ju do `app/components/tables`, a opäť použite `app/components/tables/shared` kvôli štýlovej konzistencii.

Na vykreslenie grafov sa v aplikácii používa knižnica ApexCharts, ktorá podporuje množstvo typov grafov. Všetky grafy implementované touto knižnicou sa nachádzajú v `app/components/charts`.

Na rozšírenie funkcií textového editora použite komponent `app/components/editors/text-editor` a rozširujte funkcionalitu v: `app/services/monaco-decoration.service.ts`.

Pri ďalších rozšíreniach sa riaďte základnými pravidlami Angular a dizajnovými smernicami Material Design 3.